CSCE 355: JFLAP4 — NFAs and λ-NFAs

Overview

In this assignment you will build seven nondeterministic finite automata in JFLAP: five NFAs and two λ-NFAs (NFAs with empty-string transitions, also called ε-NFAs). Each is checked automatically. The assignment is worth **30 points**.

Regular expressions below use course notation: `+` is union, `*` is Kleene star, and λ is the empty string.

Getting started

- Read the JFLAP tutorial: `https://jflap.org/tutorial/`.
- JFLAP is a free `.jar` file from `https://www.jflap.org`. You need Java 8 or higher. Start it with `java -jar JFLAP7_1.jar` (use the exact name of the file you downloaded; on macOS and Linux, upper and lower case matter).
- Unzip this assignment folder. (It is a good idea to append your username to the folder name, e.g. `JFLAP4_jsmith`.)
- In JFLAP use **File** → **Open** to open one of the provided `.jff` files. Each one is an empty *Finite Automaton*. Build your machine, then **File** → **Save** it *under the same file name*. Do not change the machine type.
- JFLAP shows the empty string as λ. Just leave it as λ; do not change it to ε. To make a λ transition, draw the transition and press Enter without typing anything.

#	Your machine (edit this file)	Test strings (do not edit)	Points
1	`problem1_01then10.jff`	`problem1_01then10_tests.txt`	3
2	`problem2_cab.jff`	`problem2_cab_tests.txt`	4.5
3	`problem3_no010.jff`	`problem3_no010_tests.txt`	4.5
4	`problem4_last_new.jff`	`problem4_last_new_tests.txt`	3
5	`problem5_10i1.jff`	`problem5_10i1_tests.txt`	3
6	`problem6_nfa_01_110_011.jff`	`problem6_nfa_01_110_011_tests.txt`	4.5
7	`problem7_enfa_01_110_011.jff`	`problem7_enfa_01_110_011_tests.txt`	7.5

NFA or λ-NFA? Rules for every machine

The test strings cannot tell an NFA from a λ-NFA, because both kinds of machine recognize the same languages. What separates them here is the machine itself, and that is checked automatically, before any test string is run:

Problem	Build a	λ transitions	Size limit
1	λ-NFA	**required**, and must matter	at most 4 states
2	NFA	**not allowed**	at most 4 states
3	NFA	**not allowed**	at most 6 states
4	NFA	**not allowed**	none
5	NFA	**not allowed**	at most 8 states
6	NFA	**not allowed**	at most 8 states
7	λ-NFA	**required**, and must matter	at most 12 edges

A machine that breaks one of these rules receives **zero credit** for that problem, even if it accepts and rejects every string correctly:

- **NFA problems (2–6): no λ transitions at all.** A single λ transition anywhere in the file earns 0 for that problem.
- **λ-NFA problems (1 and 7): use λ transitions, and make them matter.** The grader deletes all of your λ transitions and checks that the language changes. A correct machine with no λ transitions, or with λ transitions that could simply be removed, earns 0 for that problem.
- **One symbol per transition.** Every other transition reads exactly one symbol of the problem's alphabet. A label such as `01` (two symbols in a row) earns 0. Draw a separate transition for each symbol: a self-loop on `a`, `b` and `c` is three transitions, and JFLAP stacks their labels above the state. *Comma labels:* the grader reads a label such as `0,1` as one transition per symbol, and each one counts separately toward a size limit (`0,1` is two edges, `a,b,c` is three). But JFLAP itself does not: its Multiple Run reads `0,1` as a single three-character string, so your JFLAP tests will not match. Use separate transitions.
- **Exactly one initial state.**

Nondeterminism is of course allowed in both kinds of machine: a state may have several transitions on the same symbol, or none at all (a string that gets stuck is rejected, so you do not need a trap state).

Size limits. *Every* state in your file counts toward a state limit, and *every* transition (including self-loops and λ transitions, with a comma label such as `0,1` counting once per symbol) counts toward Problem 7's edge limit, whether or not it can be reached, so delete anything you are not using. Each state or edge over the limit costs **50%** of that problem's points: one over the limit earns half credit, two over earns nothing. Every limit can be met: there is a correct machine within each one.

Problems

Problem 1: λ-NFA (3 points, at most 4 states)

Construct a λ-NFA, in no more than 4 states, that recognizes any number of `01`s followed by any number of `10`s, without overlap: the language `(01)*(10)*`. For example, `010` is *not* in the

language. The alphabet is $\{0, 1\}$.

- Your machine must use λ transitions productively: deleting all of them must change the language it recognizes.
- The test file has 8 strings: the first 3 should accept, the last 5 should reject.
- The empty string should accept (see "Testing the empty string" below).

Problem 2: NFA (4.5 points, at most 4 states)

Construct an NFA (no λ transitions) in 4 or fewer states that recognizes the strings that contain `cab` as a substring: `(a+b+c)*cab(a+b+c)*`. The alphabet is $\{a, b, c\}$.

- The test file has 8 strings: the first 4 should accept, the last 4 should reject.
- The empty string should reject.
- Remember that a self-loop on `a`, `b` and `c` is three separate transitions, one per symbol.

Problem 3: NFA (4.5 points, at most 6 states)

Construct an NFA (no λ transitions) in 6 or fewer states that recognizes the strings that do not contain `010` as a substring, *or* that are just `01` repeated, that is, in `(01)*`. For example, `01010101` is accepted but `0101010` is not. The empty string should be accepted. The alphabet is $\{0, 1\}$.

- The test file has 8 strings: the first 4 should accept, the last 4 should reject.
- The empty string should accept.

Problem 4: NFA (3 points, no size limit)

Construct an NFA (no λ transitions) that recognizes the strings over $\Sigma = \{1, 2, 3, 4\}$ whose last symbol does not appear anywhere before it. For example, `1234` and `4231` are accepted; `12344` and `4214` are not.

- You may accept the empty string (it will not be tested).
- The test file has 10 strings: the first 5 should accept, the last 5 should reject.
- Remember to add a separate transition for each input symbol.

Problem 5: NFA (3 points, at most 8 states)

Construct an NFA (no λ transitions) in 8 or fewer states that recognizes the strings over $\Sigma = \{0, 1, 2\}$ that contain a substring $1\,0^i\,1$ (a `1`, then i copies of `0`, then a `1`) where $i > 0$ is a multiple of 2 or a multiple of 3, that is, $i \in \{2, 3, 4, 6, 8, 9, 10, 12, \ldots\}$:

```
(0+1+2)*1(00(00)* + 000(000)*)1(0+1+2)*
```

For example, `1001` ($i = 2$), `10001` ($i = 3$), `100001` ($i = 4$) and `21210012121` are accepted; `101` ($i = 1$), `1000001` ($i = 5$) and `2100000001` ($i = 7$) are not.

- The test file has 17 strings: the first 9 should accept, the last 8 should reject.
- The empty string should reject.

Problem 6: NFA (4.5 points, at most 8 states)

Construct an NFA, with no λ transitions and in no more than 8 states, recognizing the concatenation of any number of `01`, `110`, and/or `011`, zero or more copies of each in any order (the copies may be interleaved; look at the tests): `(01+110+011)*`. The alphabet is $\{0, 1\}$.

- The test file has 117 strings: the first 107 should accept, the last 10 should reject.
- The empty string should accept.

Problem 7: λ-NFA (7.5 points, at most 12 edges)

Construct a λ-NFA for the same language as Problem 6, `(01+110+011)*`. This is easy, actually, but this time the limit is on *edges*, not states: no more than 12 edges. Every transition counts as one edge, including self-loops and λ transitions; a label such as `0,1` counts as two edges.

- Your machine must use λ transitions productively: deleting all of them must change the language it recognizes. (A λ transition that could simply be removed does not count.)
- The test file is the same as Problem 6's: the first 107 strings should accept, the last 10 should reject.
- The empty string should accept.
- Not graded: compare your solutions to Problems 6 and 7 with respect to number of states, number of edges, clarity, etc.

Testing your machines

In JFLAP

1. Open your machine (it needs an initial state to be tested).
2. Choose **Input** $\rightarrow$ **Multiple Run**.
3. Click **Load Inputs** (at the bottom) and select the problem's test file.
4. Click **Run Inputs** and compare each row with what the problem says should accept and reject.

Testing the empty string

The test files do not contain the empty string. In Multiple Run, put the cursor on the last row and press Enter to add a blank line (an empty input), then run again. The empty string should accept in Problems 1, 3, 6 and 7, and reject in Problems 2 and 5. (Problem 4 does not test it.)

From the command line

This folder contains `test_nfa.py`, the same script that is used to grade you (Python 3 is required). From inside this folder run

```
python3 test_nfa.py
```

to check all seven problems, or `python3 test_nfa.py 3` to check only Problem 3. The report tells you, for each problem: whether the machine is the required kind and follows the rules above, which provided test strings are wrong, its size, and your score. Run it as often as you like.

Grading

Each problem earns its points only if *all* of the following hold:

- The machine is the required kind (NFA or λ-NFA) and follows the rules above.
- Every string in the problem's test file gets the expected result, and so does the empty string where the problem says what it should do.
- Your machine recognizes *exactly* the described language. The provided strings are only examples: the grader compares your machine with the described language on all strings, so a machine that passes the samples but is wrong somewhere else earns no credit. When this happens the report shows a shortest string on which your machine is wrong.

A machine that meets all of these but is over its size limit loses 50% of the problem's points per extra state (Problem 7: per extra edge).

Do not edit the test files or `test_nfa.py`. They are replaced with the original versions before grading, so changing them can only make your own testing misleading.

Submission

1. Open **YOURUSERNAME.txt** and put your university username as the *first line* of the file (only your username, nothing else on that line). This is how your submission is identified.
2. Make sure each of your seven machines is saved in its provided `.jff` file. Do not rename any of the provided files.
3. Put screenshots of all seven machines into one PDF file named `screenshots.pdf` in this folder.
4. Run `python3 test_nfa.py` one last time to make sure everything is as you expect.
5. Zip this entire folder and submit the zip as one file.